Subscription Vending with Bicep

Create a subscription under a management group, then deploy a spoke VNet peered to a central hub — with an Azure DevOps pipeline.

1. What this solution does

This Infrastructure-as-Code (IaC) package automates a common Azure landing-zone task. In a single pipeline run it performs four things in order:

- **Creates a brand-new subscription** using a subscription *alias*, and places it directly under a management group of your choice.
- **Creates a resource group** inside that new subscription.
- **Creates a spoke virtual network** (with its subnets) in that resource group.
- **Peers the spoke VNet to a central hub VNet** — both directions — so the new workload can reach shared services, on-prem, or the internet through the hub.

The whole thing is **template + variable** based: the logic lives in `.bicep` files that never change, and every environment-specific value lives in a single `.bicepparam` file. Onboarding a new landing zone means copying one parameter file and changing the values — nothing else.

2. Architecture at a glance

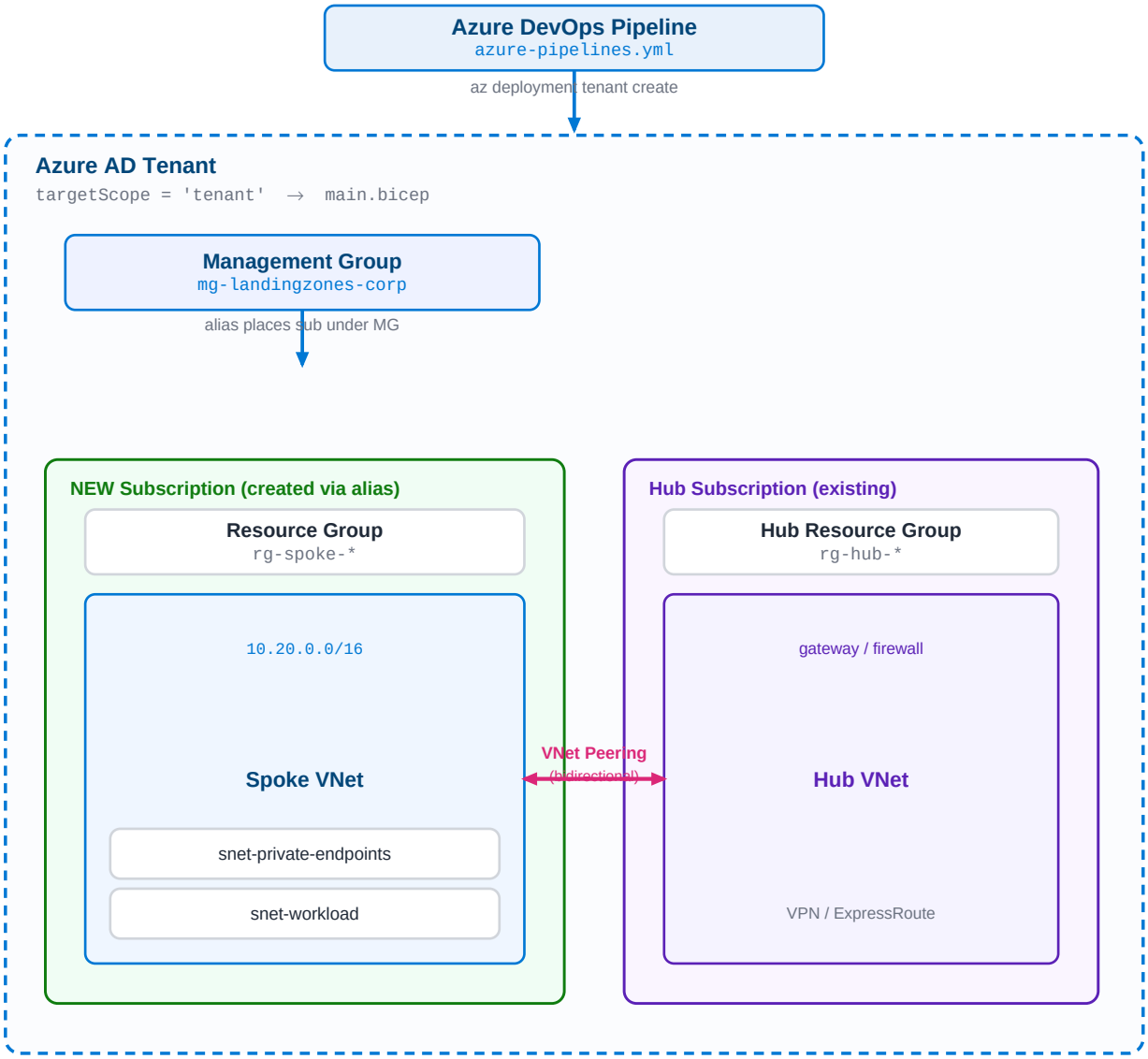

Figure 1 — The pipeline runs a tenant-scoped deployment. The subscription alias creates the new subscription under the management group; the landing-zone module then builds the RG, the spoke VNet, and both sides of the peering to the existing hub subscription.

3. Repository structure

```
repo-root/
|
|-- main.bicep                       # TENANT scope: subscription alias + landing zone
|-- bicepconfig.json                 # Bicep linter rules (optional)
|
|-- modules/
|   |-- subscription-resources.bicep # SUBSCRIPTION scope: RG + VNet + both peerings
|   |-- vnet.bicep                   # spoke VNet + variable-driven subnets
|   \-- peering.bicep                # generic one-direction peering (used twice)
|
|-- parameters/
|   |-- dev.bicepparam               # <-- copy this to add a new landing zone
|   \-- prod.bicepparam
|
\-- pipelines/
    \-- azure-pipelines.yml          # validate (what-if) + deploy
```

The nesting mirrors the Azure scope hierarchy: **tenant** (main.bicep) → **subscription** (subscription-resources.bicep) → **resource group** (vnet.bicep and peering.bicep run against the RG).

4. The code, explained file by file

main.bicep — the entry point (tenant scope)

Because creating a subscription is a tenant-level operation, this file declares `targetScope = 'tenant'`. It does two jobs:

```
resource subAlias 'Microsoft.Subscription/aliases@2021-10-01' = {
  name: subscriptionAliasName
  properties: {
    displayName: subscriptionDisplayName
    workload: subscriptionWorkload      // Production | DevTest
    billingScope: billingScope          // where the sub is billed
    additionalProperties: {
      managementGroupId: tenantResourceId(
        'Microsoft.Management/managementGroups', managementGroupId)
    }
  }
}
```

The alias is what actually provisions the subscription. Setting `managementGroupId` places it under your MG at creation time. The second job is deploying the landing zone *into* the sub that was just created:

```
module landingZone 'modules/subscription-resources.bicep' = {
  name: 'lz-${resourceGroupName}'
  scope: subscription(subAlias.properties.subscriptionId)  // new sub id
  params: { ... }
}
```

`subAlias.properties.subscriptionId` is the ID of the freshly created subscription, so the module runs inside it. This dependency is automatic.

modules/subscription-resources.bicep (subscription scope)

Runs inside the new subscription. It creates the resource group, then calls two child modules — one for the VNet and one (twice) for peering:

- **Resource group** – `Microsoft.Resources/resourceGroups`.
- **Spoke VNet** – deploys `vnet.bicep` scoped to that RG.
- **Spoke** → **Hub peering** – created in the spoke RG.
- **Hub** → **Spoke peering** – created in the hub RG (possibly another subscription) via `scope: resourceGroup(hubSubId, hubRg)`.

modules/vnet.bicep (resource-group scope)

Creates the VNet and loops over a `subnets` array, so subnets are pure data in the parameter file. Optional NSG, service endpoints and delegations are supported per subnet:

```
subnets: [for subnet in subnets: {
  name: subnet.name
  properties: {
    addressPrefix: subnet.addressPrefix
    networkSecurityGroup: (subnet.?nsgId != null) ? { id: subnet.nsgId } : null
    serviceEndpoints: subnet.?serviceEndpoints
    delegations: subnet.?delegations
  }
}]
```

modules/peering.bicep (resource-group scope)

One generic, reusable module that creates a single peering on an *existing* VNet. main's child module calls it twice — once for each direction — which is why you never write peering logic twice. The four gateway/traffic flags are parameters:

- `allowForwardedTraffic` – lets traffic that didn't originate in the peer VNet flow across (needed when the hub routes via an NVA/firewall).
- `allowGatewayTransit` (hub side) + `useRemoteGateways` (spoke side) – the pair that lets the spoke use the hub's VPN/ExpressRoute gateway. Both are driven by the single `useHubGateway` switch in the param file.

parameters/*.bicepparam — the only file you edit

Each file targets the template with `using '../main.bicep'` and supplies every value: subscription name, billing scope, management group, RG/VNet names, address space, subnets and hub details. A new landing zone = a new copy of this file.

pipelines/azure-pipelines.yml

Two stages. **Validate** runs `az bicep build` (lint) and a what-if. **Deploy** runs the tenant deployment. A runtime parameter picks the environment, which selects the matching `.bicepparam` file.

```
az deployment tenant create \
  --name lz-$(Build.BuildId) \
  --location $(location) \
  --template-file main.bicep \
  --parameters parameters/<env>.bicepparam
```

5. Step-by-step deployment

Prerequisites (one-time setup)

#	What	Why
1	ARM service connection in Azure DevOps	Lets the pipeline authenticate to Azure. Note its name.
2	Identity: Owner/Contributor on the target management group	Required for a tenant-scope deployment and to place the sub under the MG.
3	Identity: a billing role on the billing scope	EA enrollment-account owner, or MCA invoice-section owner. Authorizes subscriptio
4	Identity: Network Contributor on the hub RG/subscription	Needed to create the hub-to-spoke side of the peering.
5	ADO Environments named 'dev' and 'prod'	Referenced by the deploy job; attach a prod approval gate here.

Deploy with the pipeline (recommended)

- Push the repo (all files) to your Azure DevOps repository.
- Edit `azure-pipelines.yml`: set `azureServiceConnection` to your connection name and `location` to your region.
- Open the correct `parameters/<env>.bicepparam` and fill in your real values — especially `billingScope`, `managementGroupId`, and the hub details.
- In Azure DevOps: **Pipelines** → **New pipeline** → point at `pipelines/azure-pipelines.yml`.
- Run it, choose **dev** or **prod** from the dropdown. Validate runs first; on success, Deploy creates the subscription and the landing zone.

Deploy manually from a terminal (alternative)

```
# 1. Sign in and confirm the right tenant
az login

# 2. Lint / preview
az bicep build --file main.bicep
az deployment tenant what-if \
  --location eastus \
  --template-file main.bicep \
  --parameters parameters/dev.bicepparam

# 3. Deploy
az deployment tenant create \
  --name lz-manual \
  --location eastus \
  --template-file main.bicep \
  --parameters parameters/dev.bicepparam
```

Add a new landing zone later

- `cp parameters/dev.bicepparam parameters/<name>.bicepparam`
- Change the subscription name, MG, RG/VNet names, address space and hub details.
- Add <name> to the `values:` list in the pipeline, run, and pick it. No template changes.

6. Things to watch for

New-subscription readiness. A freshly created subscription can take a short while before its resource providers are registered. If the landing-zone step intermittently fails with ‘subscription not found’, just re-run the pipeline — it is idempotent (the alias already exists). For heavy use, split into two stages: create the sub, then run `az deployment sub create` for the network.

Non-overlapping address space. The spoke and hub VNet ranges must not overlap or Azure rejects the peering.

One-sided peering. If the identity lacks Network Contributor on the hub, only the spoke-to-hub side is created and the peering shows as *Disconnected*. Grant the hub role and re-run.

Production-grade alternative. Microsoft ships an official ‘subscription vending’ Bicep module (`br/public:lz/sub-vending`) that hardens this exact pattern with budgets, role assignments and peering orchestration.